13 - Test practic APD

Să se implementeze, **folosind Java Threads**, un program paralel cu N thread-uri în care fiecare thread generează M perechi de numere naturale (K, V) din intervalele [1, 10], respectiv [1, 100000]. La final, programul va afișa **exact** 10 perechi de forma (I, V_max_i) unde V_max_i este maximul valorilor V generate în perechile (I, V) din toate thread-urile.

Valorile M se vor alege aleator între 2 și 10 de către fiecare thread, iar rezultatul trebuie să se printeze o singură dată, la final. Implementarea va maximiza concurența între thread-uri.

Introducerea unei valori în cadrul unui thread va respecta **obligatoriu** următorul pseudocod:

- generează aleator o pereche nouă (K, V)
- printează un mesaj de forma `[Thread %d] a generat perechea (%d, %d)`
- citește valoarea maximă anterioară pentru cheia K
- așteaptă (folosind `sleep`) o secundă
- actualizează valoarea nouă de la cheia K.

În urma compilării prin intermediul comenzii `javac *.java`, va rezulta un fișier numit *Testapd.class*, care se va rula astfel: **`java Testapd N`** (unde N este numărul de thread-uri). Se presupune că programul este apelat corect și trebuie să funcționeze corect pentru orice valoare a lui N între 1 și 8 (inclusiv). Programul va trebui să afișeze mesajul de mai sus de fiecare dată când un thread generează o pereche nouă, precum și rezultatul final. Aveți mai jos un exemplu de rulare:

```
$ .javac *.java

$ java Testapd 3
[Thread 0] a generat perechea (2, 7)
[Thread 1] a generat perechea (1, 10)
[Thread 2] a generat perechea (3, 9)
[Thread 0] a generat perechea (1, 5)
[Thread 1] a generat perechea (3, 11)
[Thread 2] a generat perechea (1, 11)
[Thread 0] a generat perechea (8, 25)
[Thread 1] a generat perechea (2, 10)

(1, 11), (2, 10), (3, 11), (4, 0), (5, 0), (6, 0), (7, 0), (8, 25), (9, 0), (10, 0)
```

În exemplul de mai sus, thread-urile 0 și 1 au ales aleator M=3, iar thread-ul 2 a ales M=2. Așa cum se observă, rezultatul final conține înșiruirea celor 10 chei și valorile maxime din toate thread-urile pentru fiecare din ele. Pentru cheile fără valori (4, 6, 7, 9 și 10 din exemplul de mai sus) se consideră valoarea implicită 0.

Atenție! Nu aveți voie să folosiți ConcurrentHashMap sau ceva similar.

Atenție! Implementarea trebuie să maximizeze paralelismul. În exemplu s-au făcut 8 operații cu durata minimă de câte o secundă, însă datorită ordinii lor, toate thread-urile pot executa (chiar și cu respectarea pseudocodului de adăugare) mai multe operații la fiecare moment, deci timpul rezultat va fi (cel mai probabil) mai mic. O implementare care serializează operațiile nu va fi acceptată.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă și eventual un fișier README. Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată.